

me&u Loyalty Connector: Architecture and Deep Dive

Oct 5, 2026 · @soumya

Purpose and scope

The service lets me&u use Aura as its loyalty provider: link a guest to an Aura member, show their cashback, apply it to a cart, and record the finished order. It is Aura's implementation of me&u's "Loyalty Provider" contract, and me&u's Loyalty Connector is the caller.

What it owns

- Linking between me&u memberships and Aura members (`meu_member_linking`).
- Showing and applying one cashback offer, `aura-cashback` , which me&u treats as a `PointShopOffer` .
- Recording me&u cart events into each partner's transaction tables.
- The error and audit trail for all of the above (`meu_log` , `aura_logs`).

What it does not own

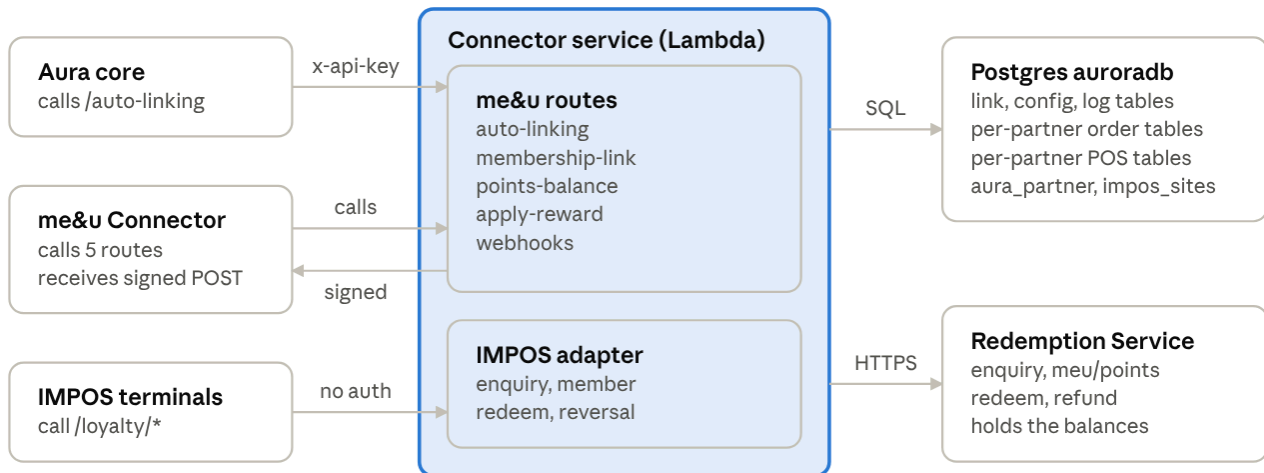
- The balance. The Redemption Service holds balances and performs every deduction.
- Earning cashback. Earn rules live elsewhere in Aura.
- Holds, expiry and refunds. Cashback is deducted the moment the guest taps Apply. If the guest later removes it or abandons the cart, returning the points is me&u's decision.
- Other offer types. `rewards` carries at most the one cashback offer; deals and promo codes are out of scope.

The same process also hosts the older IMPOS point-of-sale adapter (`/loyalty/*`). It shares the database and the Redemption Service but is unrelated to me&u.

System context

Three callers reach one Lambda service, which reads Postgres, asks the Redemption Service for balances and calls me&u back when it creates a membership.

One Lambda service connects me&u, Aura, Postgres and the Redemption Service



Highlighted box is the service this document describes. Arrows point from caller to callee.

system context · 3 callers, 3 dependencies, 1 service

Read it left to right: Aura core, me&u and the IMPOS terminals call in; the service reads and writes Postgres and talks to the Redemption Service; only `/auto-linking` makes a call out to me&u.

Trust boundaries

- me&u and Aura core are authenticated by `x-api-key` on `/auto-linking` and `/meu/webhooks` only, and the call out to me&u is signed with an HMAC.
- IMPOS terminals are not authenticated today because the signature check is switched off.
- The Redemption Service is called with no credential in this code. The Postgres login comes from the environment.

Repository, runtime and configuration

The whole service is one Express file of about 1,630 lines, built to run on AWS Lambda and also locally with `node index.js`.

Path	What it is
<code>meu-membership-auto-link/index.js</code>	The entire service: routes, helpers, SQL
<code>meu-membership-auto-link/generate-api-key.js</code>	Prints a random 32-character key for <code>API_KEY</code> . Never edits <code>.env</code> or the running server

Path	What it is
<code>meu-membership-auto-link/meu_link_status_log.sql</code>	DDL of the retired status-log table
<code>meu-membership-auto-link/package.json</code>	express 4, pg 8, axios 1, dotenv, serverless-http
<code>meu-membership-auto-link/README.md</code>	Stub only
<code>aura-api/</code>	Empty folder

Runtime

- The file exports a Lambda `handler` built with `serverless-http`. It strips the API Gateway stage prefix from the path (HTTP API v2) and sets `callbackWaitsForEmptyEventLoop = false`.
- When `NODE_ENV` is not `production` it also calls `app.listen(3000)`, so the same file serves local runs.
- One shared `pg` Pool serves simple queries. The webhook writer and the apply-reward flow check out dedicated clients because they run transactions.
- `dotenv` loads `.env` locally and does nothing in Lambda, where the environment is set on the function.

Environment variables

Variable	Used for
<code>DB_USER</code> , <code>HOST</code> , <code>DBNAME</code> , <code>DB_PASSWORD</code>	Postgres connection
<code>PORT</code>	Postgres port. The pool reads this, not <code>DB_PORT</code>
<code>API_KEY</code>	Expected <code>x-api-key</code> on <code>/auto-linking</code> and <code>/meu/webhooks</code>
<code>MEU_BASE_URL</code>	me&u Connector base URL (staging or production)
<code>MEU_PROVIDER_ID</code> , <code>MEU_SIGNING_SECRET</code>	Identify and sign outbound calls to me&u
<code>MEU_DUMMY_MODE</code>	<code>true</code> fakes me&u and balance calls for local work
<code>MEU_WITHDRAWAL_TYPE</code> (default <code>online</code>), <code>MEU_WITHDRAWAL_GATEWAY</code> (default <code>MEU</code>)	Fields sent on Redemption <code>redeem</code>

Variable	Used for
<code>MEU_LOG_BODIES</code>	<code>false</code> stops request and response bodies being logged on me&u routes

Hard-coded in code: the Redemption Service base URL (`https://jqz1xs0nr9.execute-api.ap-southeast-2.amazonaws.com/v1/`) and the offer id `aura-cashback` .

Request lifecycle and cross-cutting concerns

Every request passes the same three stages before reaching a route, and failures fall through to one shared error handler.

- Hit logger.** Wraps `res.json` and prints `[API] METHOD path -> status (ms)` for every route. For `/meu/*` and `/auto-linking` it also prints the request and response bodies with `mobile` , `phone` and `email` masked to their last three characters. Headers and query strings are never logged. `MEU_LOG_BODIES=false` turns the bodies off.
- Body parser.** `express.json` with a 5 MB limit. The raw body is kept on `req.rawBody` ; nothing uses it today.
- Route handler.** Auth is checked inside the route, not globally.
- Fallbacks.** Unknown paths return `404 {message: "API not found"}` . Anything unhandled, including malformed JSON, returns `500 {message: "Something went wrong!"}` .

Authentication

Direction	Mechanism	Where enforced
me&u or Aura core to this service	<code>x-api-key</code> header, compared in constant time (<code>crypto.timingSafeEqual</code>) with <code>API_KEY</code>	<code>/auto-linking</code> and <code>/meu/webhooks</code> only
This service to me&u	<code>x-provider-id</code> plus <code>x-signature-sha256</code> , an HMAC-SHA256 hex digest of the exact JSON body using <code>MEU_SIGNING_SECRET</code>	<code>/auto-linking</code> outbound call
IMPOS to this service	Signature middleware exists but is commented out (TODO)	Nowhere today

If `API_KEY` is not set, the key check fails closed: only `MEU_DUMMY_MODE=true` lets a request through. `/meu/membership-link`, `/meu/points-balance` and `/meu/apply-reward` perform no key check.

Dummy mode

`MEU_DUMMY_MODE=true` is one flag for the whole process. It makes `/auto-linking` skip the call to me&u and return `202` with an id like `dummy-membership-id-<ms>`. It makes balance reads return a fake value: an `externalId` of `dummy-<dollars>` gives that balance, anything else gives 12.34. An in-memory map of dummy redemptions lowers the fake balance after each dummy apply, so a local apply-then-balance sequence stays consistent. Because the flag is shared, turning it off also makes `/meu/apply-reward` call the real Redemption Service.

Logging and errors

- `storeMeuLog` writes a row to `meu_log` and never throws, so logging cannot fail a request.
- Event types in use: `MEU_AUTO_LINKING`, `MEU_AUTO_LINKING_SAVED`, `MEU_MEMBERSHIP_LINKING`, `MEU_POINTS_BALANCE`, `MEU_APPLY_REWARD`, `MEU_WEBHOOK_CART_SUBMITTED`, `MEU_WEBHOOK_CART_CLAIMED` and `MEU_WEBHOOK_LINK_BACKFILL`.
- `/meu/membership-link` also writes one `aura_logs` row per call with a status and message.
- The IMPOS routes log errors to `withdrawal_error_logs` through a separate helper.

Endpoint reference

Five me&u routes and four IMPOS routes, each documented below with what it requires, what it does and what it can return.

Route	Caller	Key check	Writes
<code>POST /auto-linking</code>	Aura core	Yes	<code>meu_member_linking</code> , <code>meu_log</code>
<code>POST /meu/membership-link</code>	me&u	No	<code>aura_logs</code> (and <code>meu_log</code> on error)
<code>POST /meu/points-balance</code>	me&u	No	Nothing on success
<code>POST /meu/apply-reward</code>	me&u	No	Redemption deduction only

Route	Caller	Key check	Writes
POST /meu/webhooks	me&u	Yes	meu_<prefix>_*, <prefix>_*, link backfill
/loyalty/* (four routes)	IMPOS	No	withdrawal_* tables via Redemption

POST /auto-linking

- **In:** { programId, payload: { memberId, externalId, venueId, mobile, firstName, lastName, email, ... } }.
- **Rejects:** 401 without a valid key; 400 if programId or payload is missing, or payload.venueId is missing.
- **Steps:** look up the active row in meu_partner_program_config for that program and venue (404 Program not configured for me&u if none). The partner_id always comes from that row, never from the payload. Then POST to me&u (or fake it in dummy mode), then save the link.
- **Save:** upsert into meu_member_linking on (program_id, member_id), setting external_id, membership_id and partner_id. It is skipped quietly if memberId, externalId or the returned id is missing. A successful save writes a MEU_AUTO_LINKING_SAVED row to meu_log.
- **Out:** me&u's own status and body, normally 202 {id}. A failed save (member not in aura_customer, or a duplicate me&u id) never changes the response; it is logged to meu_log. A me&u error status is forwarded unchanged and logged. A transport failure returns 500 {error: "Failed to auto-link membership"}.

POST /meu/membership-link

- **In:** { guestId, externalMembershipId, mobile, email, venueId, programId }.
- **Rejects:** 400 mobile is required, 400 programId is required, 404 Program not configured for me&u (active config for the program).
- **Steps:** if externalMembershipId already matches a membership_id in meu_member_linking for the program, return that row's external_id. Otherwise search aura_customer by phone or email and return the member's referral_id.
- **Out:** 200 { membershipId }, or 404 Member not found. Linking tables are only read here; nothing is inserted. Every call writes one aura_logs row, and errors also go to meu_log.

POST /meu/points-balance

- **In:** `membership.externalId`, `venueId` and `programId` are required (`400` otherwise). `card` is optional; `null`, `{}` and a missing card all mean an empty card. Other membership fields are accepted and ignored.
- **Steps:** one query for the partner and its redemption limits; one GET to the Redemption Service `meu/points`; then the offer math in the next section.
- **Out:** `{ status: "ok", membership: { id, pointsBalance, rewards } }` with the balance in cents and `rewards` holding zero or one `PointShopOffer`. `membership.id` is simply absent if it was not sent. `404` if the venue and program are not configured, or with the Redemption Service's own message if the card is not found. `500` is generic and logged.
- **Standalone:** this route shares no helper functions with any other route.

POST /meu/apply-reward

- **In:** header `x-program-id` (the program is not in the body); body `membership.externalId`, `card { venueId, items, discounts }`, `rewards.offer`.
- **Rejects:** `400` if `card.venueId` or the header is missing.
- **Silent no-op:** if there is no `externalId`, or `rewards.offer` is not exactly `aura-cashback`, the answer is `200 { status: "ok", rewards: [] }` and nothing is deducted.
- **Steps:** resolve the partner (cached five minutes), then run the apply flow described in the next section.
- **Out:** `{ status: "ok", rewards: [offer] }` with `SELECTED_TO_REDEEM` and `discountAmountInCents`, or `UNAVAILABLE_TO_REDEEM` with a cause. `404` if the balance lookup fails.

POST /meu/webhooks

- **In:** `{ type, payload }`. `401 { status: "error" }` without a valid key.
- **card-submitted and card-claimed:** run the ingestion pipeline (its own section). Answer `200 { status: "ok" }`, or `200 { status: "ok", skipped: true, message: "Membership not linked -- transaction not stored" }`. `400` if `programId`, `venue.id` or `card.id` is missing; `500` on database errors, logged to `meu_log`.
- **marketing-consent-given:** logged to the console only, answered `200`. There is no column to hold consent yet.
- **Unknown types:** answered `200` so a future event type does not error.

/loyalty/* (IMPOS adapter)

GET /loyalty/enquiry, GET /loyalty/member, POST /loyalty/redeem and POST /loyalty/reversal serve IMPOS terminals. They convert Redemption Service dollar amounts to cents (multiply by 100) and resolve the partner from `impos_sites` by `site_id` and `client_id`. As written, `client_id` is expected from the disabled signature middleware, so enquiry, member and redeem return `SITE_NOT_FOUND` until it is restored. Reversal finds the original redemption in `withdrawal_events` (by `aura_id` and `merchant_ref_trxid`) and calls Redemption `refund`.

Offer and redemption math

All amounts are in cents (1 point = 1 cent). The Redemption Service returns dollars, so its balance is multiplied by 100 and rounded, and treated as 0 if the member is not `valid`.

Inputs, read from the balance and the cart

- `balance`: the member's balance in cents.
- `applied`: sum of cart discounts whose `metadata.externalRewardId` is `aura-cashback`. This is what has already been deducted for this cart.
- `items`: sum of `items[].amountInCents`, read as line totals.
- `others`: sum of discounts that are not ours.
- `min` and `max`: the partner's limits from `partner_redemption_rule` (max of 0 means no limit).

```
applicable = \max(0, \ items - others)

spendable = balance + applied

cap = \min(spendable, \ applicable, \ max)
```

Status, first match wins

1. `spendable` is 0 or less: `UNAVAILABLE_TO_REDEEM` with `INSUFFICIENT_POINTS`.
2. `applicable` is 0 or less: `UNAVAILABLE_TO_REDEEM` with `EMPTY_CART`.
3. `cap` is below `min`: `UNAVAILABLE_TO_REDEEM` with `BELOW_MINIMUM`.
4. `applied` is above 0: `SELECTED_TO_REDEEM`, with `pointsPrice` and `discountAmountInCents` both equal to `min(applied, cap)`.
5. Otherwise: `AVAILABLE_TO_REDEEM` with `pointsPrice` equal to `cap`.

`/meu/points-balance` lists a reward only when `spendable` is above 0, so `INSUFFICIENT_POINTS` never appears there as a reward; the array is simply empty.

`/meu/apply-reward` always returns the offer, so it can show that cause.

Worked examples (balance in cents; partner limits as noted)

Case	Balance	Items	Others	Applied	Limits (min / max)	Result
No cart	2,500	0	0	0	none	<code>EMPTY_CART</code> , <code>pointsPrice</code> 2,500
\$15 cart	2,500	1,500	0	0	none	<code>AVAILABLE</code> , <code>pointsPrice</code> 1,500 (capped by the cart)
\$40 cart	2,500	4,000	0	0	none	<code>AVAILABLE</code> , <code>pointsPrice</code> 2,500 (capped by the balance)
Our \$10 already on cart	2,500	4,000	0	1,000	none	<code>SELECTED</code> , <code>discountAmountInCents</code> 1,000
\$1 cart, partner minimum \$2	2,500	100	0	0	200 / 500	<code>BELOW_MINIMUM</code> , "Minimum redemption is \$2.00"
\$40 cart, partner maximum \$5	2,500	4,000	0	0	0 / 500	<code>AVAILABLE</code> , <code>pointsPrice</code> 500
Zero balance	0	1,500	0	0	none	Balance route: <code>rewards: []</code> . Apply route: <code>INSUFFICIENT_POINTS</code>

Only one partner has a limits row today: `CORUM-202506` (minimum \$2.00, maximum \$5.00, 2 per day). Froth and Limestone have none, so they run with no minimum and no maximum. The per-day frequency fields exist in the table but the code does not read them.

Applying the reward

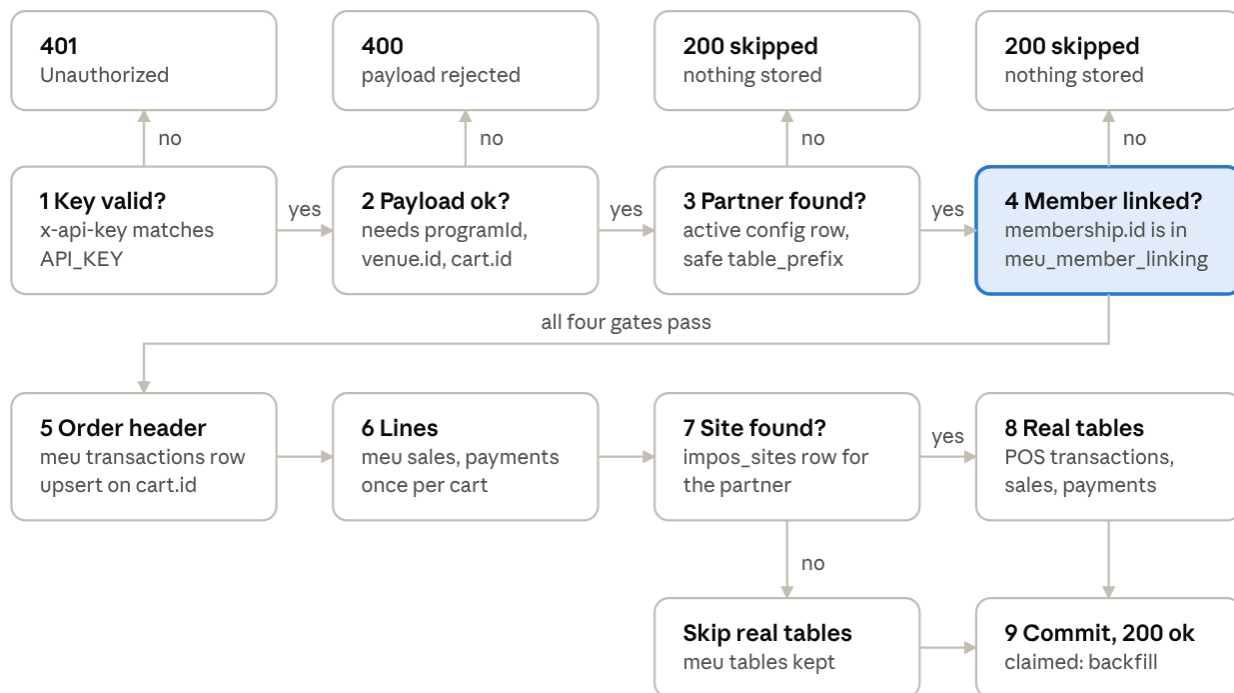
1. Take a Postgres advisory lock keyed on `externalId|partnerId`, so two quick taps run one after the other.
2. Read the balance from Redemption `enquiry` and compute the status above.
3. If there is no cause and `cap` is greater than `applied`, call Redemption `redeem` for only `cap - applied`. Re-applying an unchanged cart therefore deducts nothing.
4. If `redeem` fails and something was already applied, keep that amount; if nothing was applied, report `REDEMPTION_FAILED`.
5. Commit and return the offer.

One open question with me&u: it is not confirmed whether `items[].amountInCents` is a line total or a unit price. The code uses the lower, safer reading (line total).

Webhook ingestion pipeline

A cart event is stored only if its membership is already linked; every other event is acknowledged with a 200 and discarded.

Only linked members get their carts stored: four gates, then one transaction



Steps 3 to 8 run inside one database transaction; a skip or an error rolls it back.

The highlighted gate is the rule that decides what gets stored.

webhook ingestion · 4 gates, 4 writes, 1 transaction

Gates 1 to 4 decide whether anything is stored; steps 5 to 8 write, all inside one database transaction.

What each step does

- Key check.** 401 if `x-api-key` does not match `API_KEY`.
- Payload check.** 400 if `programId`, `venue.id` or `cart.id` is missing.
- Partner.** The active `meu_partner_program_config` row gives the partner, and `aura_partner.table_prefix` names the tables. No row, or an unsafe prefix, is a quiet skip.
- Linked member.** `membership.id` must exist in `meu_member_linking` for that program. A missing or null membership is a skip, not an error.
- Order header.** Upsert into `meu_<prefix>_transactions` keyed on `cart.id`. `cart-submitted` and `cart-claimed` share one `cart.id`, so the second event updates the same row and `event_type` records the latest.

6. **Lines.** Items go to `meu_<prefix>_sales` and discounts to `meu_<prefix>_payments`, only if that cart has none yet, so a retry adds nothing.
7. **Site.** The real tables need a `site_id`. It comes from `impos_sites`: the partner's site whose name contains "test", otherwise its lowest. With no site, the real tables are skipped and the `meu_*` rows are kept.
8. **Real tables.** `<prefix>_transactions` gets the header with `trx_raw_processed = true` and `DO NOTHING` on a key clash. `<prefix>_sales` and `<prefix>_payments` get their rows, guarded against repeats the same way, with `id` set to `MAX(id)+1`.

After the commit, `cart-claimed` also fills in `membership_id` on the link row when the member resolves from `meu_member_linking`. Retries are safe: the header stays one row and no lines are added twice.

Data model

The service owns three small me&u tables and a trio of tables per partner, and writes alongside each partner's existing POS tables. Everything lives in the `public` schema of the `auroradb` Postgres database.

me&u tables (owned by this service)

Table	Key	Purpose
<code>meu_partner_program_config</code>	<code>id</code> (bigserial)	Maps a me&u venue and program to an Aura partner: <code>partner_id</code> , <code>venue_id</code> , <code>program_id</code> , <code>active</code>
<code>meu_member_linking</code>	<code>(program_id, member_id)</code>	One row per linked member: <code>member_id</code> (uuid, an <code>aura_customer</code>), <code>external_id</code> (Aura referral id), <code>membership_id</code> (me&u id), <code>partner_id</code>
<code>meu_log</code>	<code>id</code> (bigserial)	Errors and the saved-link event: <code>program_id</code> , <code>membership_id</code> , <code>event_type</code> , <code>error_message</code> , <code>created_at</code>

`meu_member_linking` is also unique on `(program_id, external_id)` and, where set, on `(program_id, membership_id)`, so one Aura card or one me&u membership can be linked once per program. `member_id` has a foreign key to `aura_customer(member_id)` with `ON DELETE CASCADE`. That is why linking a `memberId` that is not an Aura customer fails, and is only logged.

Per-partner me&u dumps (`meu_<prefix>_transactions`, `_sales`, `_payments`)

Table	Key	Holds
<code>meu_<prefix>_transactions</code>	<code>trx_id</code> (equals <code>cart.id</code>)	One row per cart: program, membership ids, mobile, <code>venue_id</code> , <code>submitted_at</code> , last <code>event_type</code>
<code>meu_<prefix>_sales</code>	<code>sale_id</code> (uuid)	One row per cart item: <code>item_id</code> , <code>item_name</code> , <code>pos_id</code> , <code>amount_in_cents</code> , <code>quantity</code>
<code>meu_<prefix>_payments</code>	<code>payment_id</code> (uuid)	One row per cart discount: type, name, <code>amount_in_cents</code> , <code>is_internal</code> , <code>external_reward_id</code> , <code>reward_type</code> , <code>promo_code</code>

Sales and payments reference `trx_id` of their own partner's transactions table. The names are built from `aura_partner.table_prefix` after it passes a letters-digits-underscore check.

Real POS tables (`<prefix>_transactions`, `_sales`, `_payments` : 42, 49 and 51 columns)

These predate this project and are also written by the external IMPOS ingestion pipeline.

- Keys: transactions (`transaction_id`, `site_id`); sales and payments (`id`, `transaction_id`, `site_id`). Sales and payments have a composite foreign key to transactions on (`transaction_id`, `site_id`) with `ON DELETE CASCADE`.
- `transaction_id` is now `varchar` (it was `bigint`), so a me&u `cart.id` fits. The change was applied to `froth_*` and `limestone_*`.
- Required columns: `transaction_id`, `site_id`, `pos_updated_at`; `id` on sales and payments (no default); `trx_raw_processed` on transactions; `created_at` has a default.

Written column	Source
transactions: transaction_id, site_id, pos_updated_at, trx_raw_processed	cart.id ; placeholder site (see below); cart.submittedAt or now; true
sales: id, name1, quantity, item_price, pos_item_id	MAX(id)+1 ; item name; quantity; cents divided by 100; numeric posId or null
payments: id, payment_total, member_id	MAX(id)+1 ; discount cents divided by 100; the Aura member uuid

Reference tables read by the service

Table	Used for
aura_partner	table_prefix and partner identity
impos_sites	site_id choices per partner_id (Froth has 7, 6, 5 and 23; Limestone has 40 and 43)
partner_redemption_rule	Minimum and maximum redemption per partner
aura_customer	Phone or email search in /meu/membership-link ; the foreign key target of meu_member_linking.member_id
aura_logs, withdrawal_events, withdrawal_error_logs	Request audit; IMPOS reversal lookup; IMPOS error log

Partners configured today

Partner	partner_id	table_prefix	Site chosen for me&u rows
Froth	IMPOS-202607	froth	23 ("Testing Site", picked because its name contains "test")
Limestone	IMPOS-202609	limestone	40 (the lowest of 40 and 43)

Retired: meu_link_status_log is no longer written. It still holds 47 old rows.

Schema changes made during this project

- `transaction_id` changed from `bigint` to `varchar` on `froth_*` and `limestone_*`, with both composite foreign keys dropped and recreated in one transaction. Primary keys, indexes and data were unaffected (rehearsed first on a scratch schema).
- The foreign keys on `meu_limestone_sales` and `meu_limestone_payments` pointed at `meu_froth_transactions` by mistake. They now point at `meu_limestone_transactions`.

External integrations

The service talks to two outside systems: the Redemption Service for balances and deductions, and me&u's Connector for creating memberships.

Redemption Service

Base URL `https://jqz1xs0nr9.execute-api.ap-southeast-2.amazonaws.com/v1/`. The code sends no auth header. A business error (an HTTP error with a JSON body) is handed back as a normal result; only transport failures throw.

Endpoint	Used by	Sends	Fields read from the reply
GET enquiry	<code>/meu/apply-reward</code> , <code>/loyalty/enquiry</code> , <code>/loyalty/member</code>	<code>partnerId</code> , <code>barcodeText</code>	<code>success</code> , <code>valid</code> , <code>balance (dollars)</code> , <code>profile</code> , <code>errorMessage</code>
GET meu/points	<code>/meu/points-balance</code>	<code>partnerId</code> , <code>externalId</code>	<code>success</code> , <code>valid</code> , <code>balance (dollars)</code> , <code>errorCode</code> , <code>errorMessage (also</code> <code>memberId</code> , <code>barcodeText</code> , <code>totalAmount</code> , <code>currency</code> , <code>expiresAt)</code>

Endpoint	Used by	Sends	Fields read from the reply
POST redeem	/meu/apply-reward, /loyalty/redeem	partnerId, barcodeText, orderId, transactionRef, amount (cents), withdrawalType, withdrawalInstrument (Halo_Loyalty_Card), tipAmount (0), withdrawalGateWay	success, amountRedeemed, remainingBalance (dollars), partnerReference, errorCode, errorMessage
POST refund	/loyalty/reversal	original redemption details and amount	success, remainingBalance

For me&u redemptions the code sends `withdrawalType` `online` and gateway `MEU`; IMPOS sends `instore` and `IMPOS`. Whether the Redemption Service accepts the me&u values is marked as a TODO. `meu/points` has so far been seen only through its not-found reply (`CARD_NOT_FOUND`, "No member is mapped with the barcode/card").

me&u Connector, outbound

- POST `{MEU_BASE_URL}/v0/membership-programs/{programId}/memberships` with the `/auto-linking` payload as the body.
- Headers: `x-provider-id`, `x-signature-sha256` (HMAC-SHA256 hex of the exact body string) and `Content-Type: application/json`.
- The reply carries `id`, the me&u membership id, which is saved as `membership_id`.

me&u Connector, inbound

- me&u calls `membership-link`, `points-balance`, `apply-reward` and the webhooks. Its documentation lists an `X-Api-Key` header on these; this service enforces it only on the webhooks and `/auto-linking`.
- `apply-reward` also receives `x-program-id` (and, per me&u's docs, a signature header that this service does not verify).
- The reward object returned to me&u carries `id`, `type: PointShopOffer`, `name`, `description`, `pointsPrice`, `status`, and, depending on status, `nonRedeemableCause` or `discountAmountInCents`. Optional fields me&u allows (`code`, `image`, `validFrom`, `validUntil`, `redeemedAt`, `targetVenueId`, `targetVenueIds`) do not apply to a single cashback offer and are left out.

Resolution chain and onboarding a partner

Every me&u request is turned into Aura identities by the same chain, so onboarding a partner is mostly adding the right rows and tables.

The chain

1. me&u sends `venue.id` (or `venueId`) and `programId`.
2. `meu_partner_program_config` (active rows only) gives the Aura `partner_id`.
3. `aura_partner` gives that partner's `table_prefix`, which names the tables (`meu_<prefix>_*` and `<prefix>_*`). The prefix must match letters, digits and underscores, starting with a letter, before it is placed into SQL text. Table names cannot be query parameters, so this check is what prevents injection.
4. `partner_redemption_rule` gives the minimum and maximum redemption, if a row exists.
5. `impos_sites` gives the `site_id` for the real POS tables: the site whose name contains "test" if there is one, otherwise the lowest `site_id`.

Onboarding a new partner

- ☐ `aura_partner` row with a `table_prefix` (for example `acme`).
- ☐ Real POS tables `acme_transactions`, `acme_sales`, `acme_payments` exist, with `transaction_id` as `varchar` and the same composite keys as Froth.
- ☐ me&u tables `meu_acme_transactions`, `meu_acme_sales`, `meu_acme_payments`, with the foreign keys pointing at `meu_acme_transactions` (the Limestone set was first copied from Froth and pointed at the wrong table).
- ☐ `meu_partner_program_config` row: Aura `partner_id`, me&u venue id, program id, `active = true`.
- ☐ At least one `impos_sites` row for the partner. Without it the real-table write is skipped and only the `meu_*` tables fill.
- ☐ Optional `partner_redemption_rule` row for minimum and maximum.
- ☐ Link a test member through `/auto-linking`, send a test `cart-submitted` webhook, and confirm rows in both table sets and no new rows in `meu_log`.

No restart is needed for config changes, except that `/meu/apply-reward` caches the partner lookup in memory for five minutes, so a deactivated venue can keep working there for up to that long.

Testing and verification

The service is tested against the real local database with me&u and the Redemption Service replaced by mocks, so no test ever calls either real system.

How to run it

- Locally: `node index.js` serves on port 3000 with `.env` loaded. `MEU_DUMMY_MODE=true` is the safe default.
- Dummy balances: an `externalId` of `dummy-25` gives a \$25 balance. Anything else gives \$12.34.
- Going live for balance reads: set `MEU_DUMMY_MODE=false` and restart. The flag is process-wide, so apply-reward then also hits the real Redemption Service.

Test harness

- Scripts preload a mock with `node -r`. The me&u mock replaces `axios.post`, answers membership creation and records what would have been sent. The Redemption mock replaces `axios.get` and `axios.post`.
- The mocks also move `app.listen` to a test port, so a dev server on 3000 is left alone.
- Each run records baselines, exercises the routes, deletes only what it created, then re-queries the tables to prove the cleanup by content, not by row count.

What has been verified live

Area	Checks
Linking and <code>meu_log</code>	20 checks: first link, re-link, unconfigured program, missing payload, missing <code>externalId</code> , member not in <code>aura_customer</code> , duplicate me&u id, me&u 409, dummy mode; the retired table receives nothing
Webhook ingestion	Froth and Limestone: header, item and discount rows land in the <code>meu_*</code> tables and, once a site resolves, in the real tables; a repeated webhook adds no rows
Points balance	<code>EMPTY_CART</code> , <code>AVAILABLE</code> , <code>SELECTED</code> ; <code>cart</code> as <code>null</code> , <code>{}</code> or missing; minimal body; real-mode URL assertion for <code>meu/points</code> through a mocked client
Schema work	The <code>transaction_id</code> type change rehearsed on a scratch schema before touching real tables

Gotchas found while testing

- Comments or trailing commas in a JSON body give `500 Something went wrong!` from the global handler, not a `400`.
- `/auto-linking` and `/meu/webhooks` need the `x-api-key` header in every test call.
- Test data must use members and programs that do not hold real link rows; one older script deletes link rows across all programs, so it is not safe to rerun blindly.

Key decisions and why

The current shape comes from a handful of deliberate choices; each is listed with its reason so a later change can be judged against it.

Decision	Why
Cashback is deducted immediately when the guest applies it; no hold or expiry	Keeps Aura simple and stateless on this path. Giving points back after a removed reward or abandoned cart is me&u's responsibility
One fixed offer id, <code>aura-cashback</code> , shared by both directions	me&u echoes the id back on every cart, so it is the contract that lets Aura recognise its own discount
<code>partner_id</code> always comes from <code>meu_partner_program_config</code> , never from the request	A caller must not be able to name which partner it acts as
Webhook orders are stored only for members already in <code>meu_member_linking</code>	Keeps the transaction tables limited to known Aura members
Webhook member lookup uses <code>meu_member_linking</code> only, not <code>aura_customer</code>	The webhook path should trust links Aura created, not a phone or email match
Per-partner tables (<code>meu_<prefix>_*</code>) instead of one shared set	Mirrors the existing per-partner POS convention and keeps partners apart
<code>/meu/points-balance</code> is standalone and reads the dedicated <code>meu/points</code> endpoint	Display logic changes independently of the apply flow; no shared helpers to break
Key check only on <code>/auto-linking</code> and <code>/meu/webhooks</code>	The chosen scope for now, even though me&u's docs list the header on the other routes
<code>meu_log</code> replaces the old link status table	One place for the audit and error trail

Decision	Why
Real-table header insert uses <code>DO NOTHING</code> ; the <code>meu_*</code> header uses <code>DO UPDATE</code>	The real tables are also written by the POS pipeline, so a key clash must never overwrite its row
Real-table <code>id</code> is <code>MAX(id)+1</code> and <code>site_id</code> is a per-partner placeholder	Interim answers: those columns have no default or no me&u source yet

Limitations, risks and open items

The items that matter most are the three that touch shared data or money: the real-table `id` race, the placeholder `site_id` , and the missing key check on the apply route.

Item	Impact	Suggested direction
Real-table <code>id</code> is <code>MAX(id)+1</code>	Two concurrent writers to the same table could pick the same <code>id</code>	Give those columns an identity or sequence, or lock during the insert
<code>site_id</code> is one placeholder per partner (Froth 23, Limestone 40)	Every me&u order is attributed to one site regardless of the real venue	Map each me&u venue to a real site, for example a <code>site_id</code> column on <code>meu_partner_program_config</code>
Real tables are shared with the external IMPOS pipeline	Downstream readers of <code>froth_*</code> and <code>limestone_*</code> may not expect me&u rows or the <code>varchar transaction_id</code>	Confirm consumers; consider a source marker column
No key check on <code>membership-link</code> , <code>points-balance</code> , <code>apply-reward</code>	Anyone who can reach the URL can read balances or trigger a deduction	Add <code>meuAuth0k</code> to these routes (me&u's docs list the header)
<code>/loyalty/*</code> signature check is commented out	Enquiry, member and redeem return <code>SITE_NOT_FOUND</code> ; no auth	Restore the verifier file
Sales and payments dedup is check-then-insert	Two simultaneous duplicate deliveries could both insert	Unique constraint on (<code>trx_id</code> , <code>item</code>) or a row lock per cart
First <code>cart-claimed</code> for an unlinked member always skips	The gate runs before the link backfill that could have linked them	Run the backfill before the gate
Skip message always says "Membership not linked"	Also shown when the partner is unresolved	Return distinct messages

Item	Impact	Suggested direction
Offer id and text are hard-coded	Cannot differ per partner or change without a deploy	Add <code>offer_id</code> , <code>offer_name</code> , <code>offer_description</code> to <code>meu_partner_program_config</code> ; treat the id as write-once
<code>PORT</code> is read as the DB port	Surprising: <code>DB_PORT</code> is ignored; Lambda must set <code>PORT</code>	Read <code>DB_PORT</code>
Malformed JSON returns 500	Looks like a server fault	Return 400 for parse errors
<code>MEU_DUMMY_MODE</code> is one process-wide flag	Going live for balances also makes apply-reward live	Separate flags per route group
Marketing consent is not stored	The event is acknowledged and dropped	Add a consent column or table
<code>items[].amountInCents</code> meaning unconfirmed	Cart total may be understated if it is a unit price	Confirm with me&u
<code>withdrawalType</code> <code>online</code> and <code>gateway</code> MEU unconfirmed	<code>redeem</code> may reject them	Confirm with the Redemption Service team
<code>meu/points</code> not yet exercised live	Only the not-found reply has been seen	Call it with a real card in a safe window
Leftovers	<code>meu_link_status_log</code> (47 rows) and old test carts in <code>meu_froth_transactions</code>	Archive or drop when ready
<code>created_at</code> on <code>meu_member_linking</code> and <code>meu_partner_program_config</code> is typed <code>time</code>	Dates are lost	Change to <code>timestamp</code>